

Horizontal Pod Autoscaler no Kubernetes

Checkpoint - Cloud Computing, 2º Semestre

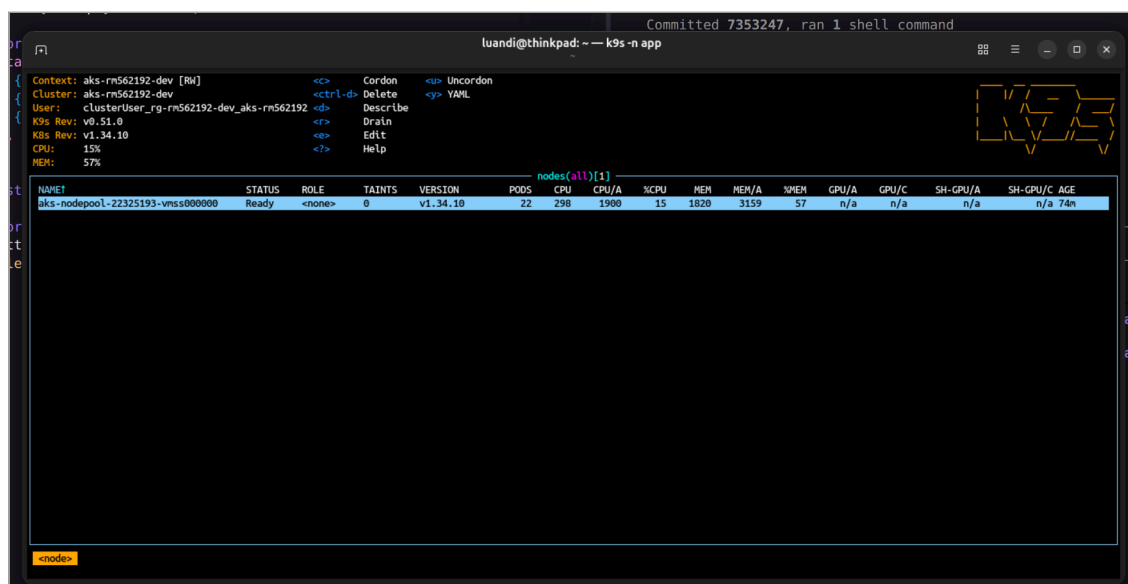
Equipe

Nome	RM
Anderson Huang	rm565920
Bruno Henrique	rm566277
Ronaldo Attamah	rm564630
Luiz Brito	rm562192
Guyllherme Miguel	rm562374

Ambiente utilizado

Usamos Azure (AKS) em vez da VM DEVASC. O cluster foi provisionado com Terraform (módulo Azure/aks/azurerms, versão do Kubernetes 1.34) e a aplicação foi sincronizada por GitOps com FluxCD, apontando direto pra pasta fluxcd/nginx/ do repositório. A aplicação de exemplo é o Nginx, exposta via Service do tipo LoadBalancer com IP público da Azure.

Evidências



```
luandi@thinkpad: ~ — k9s - n app
Committed 7353247, ran 1 shell command

Context: aks-rm562192-dev [RM]
Cluster: aks-rm562192-dev
User: clusterUser_rg-rm562192-dev_aks-rm562192
K9s Rev: v0.51.0
K9s Rev: v1.34.10
CPU: 15%
MEM: 57%

nodes(all)[1]
NAME STATUS ROLE TAINTS VERSION PODS CPU CPU/A %CPU MEM MEM/A %MEM GPU/A GPU/C SH-GPU/A SH-GPU/C AGE
aks-nodepool-22325193-vmss000000 Ready <none> 0 v1.34.10 22 298 1900 15 1820 3159 57 n/a n/a n/a n/a 74m
```

Cluster Kubernetes funcionando: nó Ready, versão 1.34.10, 22 pods em execução (k9s, view de nodes)

```
luandi@thinkpad: ~ — k9s -n app
Context: aks-rm562192-dev [RM]
Cluster: aks-rm562192-dev
User: clusterUser_rg-rm562192-dev_aks-rm562192
K9s Rev: v0.51.0
K8s Rev: v1.34.10
CPU: 11%
MEM: 57%

Deployments (app) [1]
NAME    READY    UP-TO-DATE    AVAILABLE    AGE
nginx    2/2      2              2            68m
```

Deployment do Nginx criado no namespace app, 2/2 réplicas prontas

```
luandi@thinkpad: ~ — k9s -n app
Context: aks-rm562192-dev [RM]
Cluster: aks-rm562192-dev
User: clusterUser_rg-rm562192-dev_aks-rm562192
K9s Rev: v0.51.0
K8s Rev: v1.34.10
CPU: 11%
MEM: 57%

Pods (app) [2]
NAME                                PF    READY    STATUS    RESTARTS    CPU    %CPU/R    %CPU/L    MEM    %MEM/R    %MEM/L    IP            NODE                                AGE
nginx-7f68fc9ddf-gqkbb              ●    1/1      Running    0            1      2          1         4      7          3    10.0.1.14    aks-nodepool-22325193-vms000000    47m
nginx-7f68fc9ddf-wzknr              ●    1/1      Running    0            1      2          1         4      6          3    10.0.1.23    aks-nodepool-22325193-vms000000    47m
```

Pods em execução antes do teste de carga, CPU em torno de 1-2m (praticamente ocioso)

```
luandi@thinkpad: ~ — k9s -n app
Context: aks-rm562192-dev [RM]
Cluster: aks-rm562192-dev
User: clusterUser_rg-rm562192-dev_aks-rm562192
K9s Rev: v0.51.0
K8s Rev: v1.34.10
CPU: 11%
MEM: 57%

NAME      REFERENCE      TARGETS      MINPODS      MAXPODS      REPLICAS      AGE
-----      -
nginx      Deployment/nginx  cpu: 2%/50%      2             10            2             68m

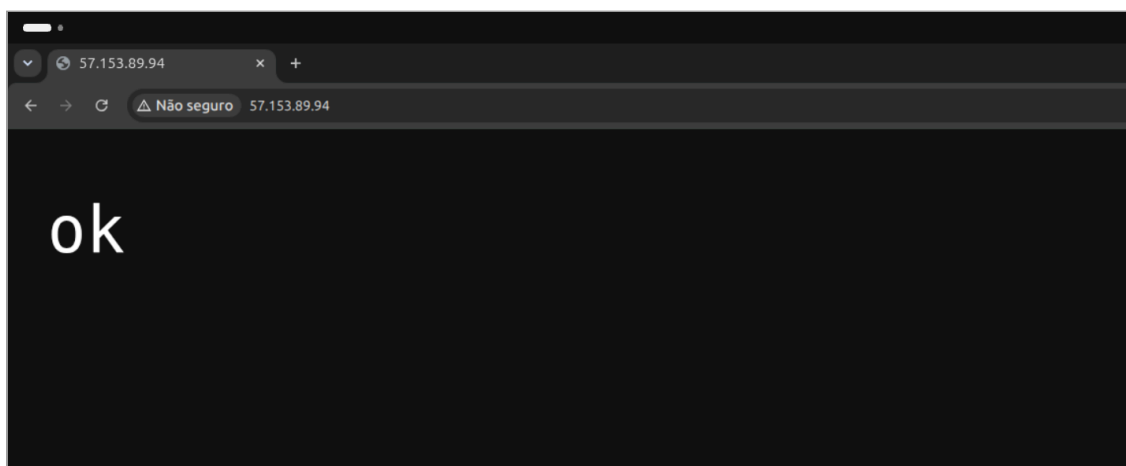
horizontalpodautoscalers(app)[1]
```

HPA configurado: mínimo 2, máximo 10 réplicas, alvo de CPU em 50%

```
luandi@thinkpad: ~ — k9s -n app
Context: aks-rm562192-dev [RM]
Cluster: aks-rm562192-dev
User: clusterUser_rg-rm562192-dev_aks-rm562192
K9s Rev: v0.51.0
K8s Rev: v1.34.10
CPU: 10%
MEM: 57%

Describe(app/nginx)
Name: nginx
Namespace: app
Labels: kustomize.toolkit.fluxcd.io/name=flux2-sync, kustomize.toolkit.fluxcd.io/namespace=flux-system
Annotations: <none>
CreationTimestamp: Mon, 14 Sep 2026 09:41:51 -0300
Reference: Deployment/nginx
Metrics: ( current / target )
  resource cpu on pods (as a percentage of request): 2% (1n) / 50%
Min replicas: 2
Max replicas: 10
Behavior:
  Scale Up:
    Stabilization Window: 30 seconds
    Select Policy: Max
    Policies:
      - Type: Pods Value: 1 Period: 15 seconds
  Scale Down:
    Stabilization Window: 30 seconds
    Select Policy: Max
    Policies:
      - Type: Pods Value: 1 Period: 15 seconds
Deployment pods: 2 current / 2 desired
Conditions:
```

Describe do HPA, mostrando a métrica atual e a política de scale up/down configurada



Aplicação respondendo pelo IP público do LoadBalancer

A execução do teste de carga, o comportamento dos pods durante o aumento de carga e a quantidade de réplicas após a atuação do HPA estão registrados no vídeo entregue junto com este PDF.

Perguntas

O que é o Horizontal Pod Autoscaler?

É um controlador do Kubernetes que ajusta sozinho o número de réplicas de um Deployment com base em métricas de uso, normalmente CPU ou memória. Ele fica checando periodicamente se o consumo está acima ou abaixo de um alvo definido e sobe ou desce o número de pods pra manter esse alvo, sem precisar de alguém rodando `kubectl scale` na mão.

Qual métrica foi utilizada para realizar o escalonamento?

Utilização de CPU (`averageUtilization`), calculada em cima do valor de `request` de CPU definido no container do Nginx.

Qual foi o número mínimo e máximo de réplicas utilizado pela equipe?

Mínimo de 2 réplicas e máximo de 10, seguindo a configuração de referência do roteiro.

O que aconteceu com a quantidade de pods quando a carga sobre a aplicação aumentou?

No teste que rodamos, a CPU saiu de cerca de 2% (praticamente ocioso) pra 137% do alvo de 50% assim que a carga bateu no endpoint que criamos pra consumir CPU de verdade. O HPA reagiu subindo as réplicas de 2 pra 3 e depois pra 4 em pouco mais de meio minuto, seguindo a política de no máximo um pod novo a cada 15 segundos que definimos.

O que aconteceu após a redução ou encerramento da carga?

Assim que o teste de carga terminou, a CPU caiu de novo abaixo do alvo. Com a janela de estabilização configurada em 30 segundos (bem mais curta que o padrão de 5 minutos do Kubernetes), o HPA já começou a reduzir as réplicas pouco depois, voltando ao mínimo de 2 em menos de dois minutos.

Qual é a função do Metrics Server no funcionamento do HPA?

O Metrics Server coleta o uso de CPU e memória de cada pod e node a partir do kubelet e expõe esse dado pra API do Kubernetes. Sem ele o HPA não tem como

saber quanto de CPU cada pod está usando, então não teria em cima de que decidir se escala ou não.

Qual seria a vantagem de utilizar HPA em uma aplicação executada em um ambiente de nuvem?

A principal vantagem é ajustar a capacidade automaticamente conforme a demanda real. Isso evita pagar por réplicas paradas em horário de baixo uso e evita também ficar sem capacidade suficiente num pico de acesso, o que numa nuvem faz sentido porque dá pra provisionar e desprovisionar recursos rápido, diferente de um servidor físico.

Quais dificuldades foram encontradas durante a atividade e como foram resolvidas?

- A versão 1.33 do Kubernetes não estava disponível na região westeurope fora do plano de suporte de longo prazo. Resolvemos fixando a versão 1.34 no Terraform.
- Um terraform apply caiu no meio da criação do cluster. Resolvemos só rodando o apply de novo.
- No começo deixamos o request de CPU do Nginx bem baixo (10m, depois 30m) achando que ia deixar o teste mais rápido de disparar. Só que o próprio pico de CPU do boot do Nginx já estourava 50% desse request, e isso criava um loop: subia pod novo, o boot desse pod novo gerava outro pico, o HPA subia mais um pod, e assim até bater no teto de 10 sem carga real nenhuma. Voltamos o request pro valor original de 50m e o problema sumiu.
- As ferramentas sugeridas no roteiro (Apache Benchmark, Siege, wrk) deram resultado confuso de ler, difícil de tirar print que fizesse sentido. Trocamos pelo k6, que já sai com um resumo organizado de requests por segundo, latência e taxa de erro.
- Mesmo com o k6 gerando bastante requisição, o Nginx servindo só a página estática padrão não consumia CPU suficiente pra passar dos 50% configurados. Resolvemos adicionando um endpoint próprio (/load) que roda um cálculo pesado a cada requisição, usando a imagem OpenResty (que é o próprio Nginx com suporte a Lua compilado junto, não é outra aplicação).
- O binário do k6 instalado via snap simplesmente não rodava quando chamado de forma automatizada, sem dar nenhum erro visível. Descobrimos pelo log do sistema que era o AppArmor bloqueando o snap de escrever a saída redirecionada. Resolvemos baixando o binário oficial do k6 direto do GitHub, fora do snap.

Conclusão

O HPA funcionou como esperado. Parado, a aplicação ficou nas 2 réplicas mínimas com CPU bem baixa. Sob carga real de CPU, ele reagiu em poucos segundos e escalou até 4 réplicas de forma gradual, e assim que a carga parou voltou pro mínimo também de forma gradual, sem oscilar. O ajuste do behavior no HPA foi importante pra conseguir ver esse ciclo completo (subida e descida) dentro de um teste curto, porque o comportamento padrão do Kubernetes demora até 5 minutos pra começar a reduzir réplicas depois que a carga cai.

Repositório: github.com/luizbrito7/cp03-scalability.fiap